

Code Explanation

Code Explanation: AWS Glue PySpark ETL Job
This code implements a production-ready ETL pipeline for customer order analytics with SCD Type 2 (Slowly Changing Dimension) support using Apache Hudi.---

****High-Level Flow****

1. Initialize Spark/Glue contexts
2. Load configuration from YAML
3. Read customer and order data from S3
4. Clean data (remove nulls, duplicates)
5. Write cleaned data to catalog
6. Apply SCD Type 2 logic and write to Hudi
7. Aggregate customer spending metrics
8. Register all outputs to Glue Data Catalog---

****Step-by-Step Breakdown****

****1. Context Initialization (`initialize\_spark\_contexts`)****

****Purpose:**** Set up Spark and Glue execution environments
****Logic:****- Checks if running in existing Spark context (test mode)- If not, creates new SparkSession with Glue-optimized configs: - Kryo serialization for performance - Hive metastore integration disabled - Log level set to WARN- Attempts to initialize GlueContext (fails gracefully in local mode)
****Returns:**** Tuple of (SparkSession, GlueContext or None)---

****2. Configuration Loading (`get\_job\_parameters`)****

****Purpose:**** Load job parameters from external YAML file
****Logic:****- Constructs path to `config/glue\_params.yaml` relative to script location- Parses YAML into dictionary containing: - Input/output S3 paths - Data formats - Catalog database/table names - Processing flags (SCD2, aggregation) - Hudi configuration
****Returns:**** Dictionary with all job parameters---

****3. S3 Access Validation (`validate\_s3\_access`)****

****Purpose:**** Pre-flight check for S3 path accessibility
****Logic:****- Validates S3 URI format (must start with `s3://`)- In production, would attempt bucket listing- Currently returns True for valid-format paths
****Returns:**** Boolean indicating accessibility---

****4. Data Reading (`DataReader` class)****

****Purpose:**** Encapsulate safe data reading with format dispatch
****Methods:****
- `_read_csv`: Reads CSV with header- Applies schema if provided, otherwise infers- Returns DataFrame
- `_read_parquet`: Reads Parquet files directly- Returns DataFrame
- `read_data` (public)`: Dispatches to appropriate reader based on format_type- Raises ValueError for unsupported formats---

****5. Data Writing (`DataWriter` class)****

****Purpose:**** Encapsulate safe data writing with format dispatch
****Methods:****
- `_write_parquet`: Writes DataFrame as Parquet- Supports overwrite/append modes
- `_write_csv`: Writes DataFrame as CSV with header- Supports overwrite/append modes
- `_write_hudi`: Writes to Hudi table with provided options- Always uses append mode (Hudi manages upserts)
- `write_data` (public)`: Dispatches to appropriate writer- Validates Hudi options if format is Hudi---

****6. Safe Read Wrapper (`read\_data\_safe`)****

****Purpose:**** Read data with error handling and normalization****Logic:****1. Validates S3 access2. Uses DataReader to load data3. Normalizes all column names to lowercase4. Returns None on any error (with logging)****Returns:**** DataFrame or None---

****7. Safe Write Wrapper (`write\_data\_safe`)****

****Purpose:**** Write data with error handling****Logic:****1. Validates S3 access2. Uses DataWriter to save data3. Logs success/failure4. Returns boolean status****Returns:**** True if successful, False otherwise---

****8. Schema Definitions****

****`get\_customer\_schema`:**** Defines customer table structure: - custid (String, not null) - name (String, not null) - emailid (String, not null) - region (String, not null)****`get\_order\_schema`:**** Defines order table structure: - orderid (String, not null) - itemname (String, not null) - priceperunit (Decimal(10,2), not null) - qty (Integer, not null) - date (Date, not null) - custid (String, not null)---

****9. Data Cleaning (`clean\_dataframe`)****

****Purpose:**** Remove invalid and duplicate records****Logic:****1. ****Drop actual NULLs:**** Uses `dropna()` to remove rows with any null values2. ****Drop string nulls:**** Filters out rows where any column equals values in `null\_string\_values` list (e.g., "Null", "NULL")3. ****Remove duplicates:**** Uses `dropDuplicates()` to keep only first occurrence****Returns:**** Cleaned DataFrame---

****10. SCD Type 2 Implementation (`apply\_scd2`)****

****Purpose:**** Create historical tracking for customer-order relationships****Logic:********Step 1 - Join Data:**- Inner join orders with customers on custid- Selects relevant columns from both tables****Step 2 - Add SCD2 Metadata:****- `isactive`: Boolean flag (True for current records)- `startdate`: Timestamp when record became active- `enddate`: NULL for active records (set when superseded)- `opts`: Timestamp for optimistic locking/versioning****Returns:**** DataFrame with SCD2 tracking columns****Note:**** This creates initial records. Hudi handles updates/versioning on subsequent runs.---

****11. Customer Spending Aggregation (`aggregate\_customer\_spend`)****

****Purpose:**** Calculate spending metrics per customer per date****Logic:********Step 1 - Calculate Total Spend:**- Adds `totalspend` column: `priceperunit \* qty`****Step 2 - Group and Aggregate:****- Groups by: custid, name, region, date- Calculates: - `total\_spend`: Sum of all order totals - `order\_count`: Number of orders - `total\_quantity`: Sum of all quantities - `max\_price`: Highest price per unit - `min\_price`: Lowest price per unit****Returns:**** Aggregated DataFrame with spending metrics---

****12. Catalog Registration (`register\_table\_to\_catalog`)****

****Purpose:**** Make data queryable via Glue Data Catalog****Logic:****1. Creates database if it doesn't exist2. Writes DataFrame as Parquet to S3 path3. Registers as managed table in catalog4. Enables SQL queries via Athena/Spark SQL****Returns:**** Boolean success status---

****13. Main ETL Orchestration (`main`)****

****Purpose:**** Execute complete ETL pipeline****Execution Flow:********Phase 1 - Setup:**- Initialize Spark/Glue contexts- Load YAML configuration- Extract all parameters****Phase 2 - Ingestion:****- Read

customer data with schema validation- Read order data with schema validation- Log record counts**Phase 3 - Cleaning:**- Clean customer data (remove nulls/duplicates)- Clean order data (remove nulls/duplicates)- Log cleaned record counts**Phase 4 - Catalog Population:**- Write cleaned customer data to S3 as Parquet- Register customer table in catalog- Write cleaned order data to S3 as Parquet- Register order table in catalog**Phase 5 - SCD2 Processing (if enabled):** - Apply SCD Type 2 logic to joined data- Configure Hudi options from parameters- Write order summary to Hudi table (enables time-travel queries)**Phase 6 - Aggregation (if enabled):** - Calculate customer spending metrics- Write aggregated data to S3 as Parquet- Register aggregate table in catalog**Phase 7 - Completion:**- Log success message- Exit cleanly---

****Key Design Patterns****

1. ****Separation of Concerns:**** Reader/Writer classes isolate I/O logic2. ****Fail-Safe Operations:**** All I/O wrapped in try-except with None/False returns3. ****Configuration-Driven:**** All paths/settings externalized to YAML4. ****Schema Enforcement:**** Explicit schemas prevent data type issues5. ****Idempotent Writes:**** Overwrite mode ensures repeatable runs6. ****Hudi Integration:**** Enables incremental processing and time-travel queries---

****Error Handling Strategy****

- S3 access validated before operations- Read failures return None (job can decide to continue/abort)- Write failures return False (logged but don't crash job)- Missing GlueContext handled gracefully (enables local testing)- Invalid formats raise ValueError (fail-fast for config errors)